

操作系统实验文档

Lab 11 - CPU调度 CPU Scheduling

eb4d26b1

Built by github-actions, at 2025-04-28 16:45:12+08:00

1. 每周实验

1.1 CPU Scheduling

1.1.1 实验目的

1. 掌握 xv6 调度过程
2. 掌握 Round-Robin 调度算法的具体实现

🚩 xv6-lab9 代码分支

<https://github.com/yuk1i/SUSTech-OS-2025/tree/xv6-lab9>

使用命令 `git clone https://github.com/yuk1i/SUSTech-OS-2025 -b xv6-lab9 xv6lab9` 下载 xv6-lab9 代码。

使用 `make run` 运行本次 Lab 的内核，它会启动第一个用户进程 `init`，`init` 会启动 Shell 进程 `sh`。在 `sh` 中输入 `schedtest` 运行用户程序观察运行结果。

在本次的实验情景中，我们将启用时钟中断，时钟中断会按照设定的间隔时间触发时钟中断，时钟中断处理函数将调度到下一个用户进程，多个用户进程按照 Round-Robin 轮流运行，每个进程每次执行一个时间片的时间，直到运行结束。

1.1.2 时钟中断

我们在 Week4 的实验课上已经了解过什么是时钟中断。

需要注意的一点是，我们需要每隔一定时间就触发一次时钟中断，但是通过 `sbi_set_timer()` 接口每次只能设置一次时钟中断。所以我们在初始化的时候设置第一次时钟中断，在每一次时钟中断的处理中，设置下一次的时钟中断。

时钟中断的准备工作，包括时钟中断的初始化 `timer_init()`，和下一次时钟中断的设置 `set_next_timer()`：

```
//os/timer.c

/// Enable timer interrupt
void timer_init() {
    // Enable supervisor timer interrupt
    w_sie(r_sie() | SIE_STIE);
    set_next_timer();
}
```

```
// /// Set the next timer interrupt
void set_next_timer() {
    const uint64 timebase = CPU_FREQ / TICKS_PER_SEC;
    if (on_vf2_board) {
        set_timer(get_cycle() + timebase);
    } else {
        w_stimecmp(r_time() + timebase);
    }
}
```

操作系统启动后，会进行调用 `timer_init()` 使能时钟中断，并设置第一次时钟中断：

```
//os/main.c

static void bootcpu_init(){
    //.....

    timer_init();

    //.....
}
```

在用户进程执行的过程中，一旦触发时钟中断，会跳转到trap处理流程的 `usertrap()` 进行时钟中断的处理。

在时钟中断处理时，设置下一次时钟中断，并且将`which_dev`设为1，1代表需要进行调度。之后在trap 处理函数的最后判断`which_dev`的值，如果为1，则调用 `yield()` 让出 cpu 资源，调度到下一个进程。

```
//os/trap.c

void usertrap() {
    //.....

    if (cause & SCAUSE_INTERRUPT) {
        which_dev = handle_intr();
    }
    //.....

    // if it's a timer intr, call yield to give up CPU.
    if (which_dev == 1)
        yield();
    //.....
}

static int handle_intr(void) {
    uint64 cause = r_scause();
    uint64 code = cause & SCAUSE_EXCEPTION_CODE_MASK;
    if (code == SupervisorTimer) {
        tracef("time interrupt!");
        if (cpuid() == 0) {
            acquire(&tickslock);
            ticks++;
            wakeup(&ticks);
            release(&tickslock);
        }
    }
}
```

```

    }
    set_next_timer();//set next timer
    return 1;
} else if (code == SupervisorExternal) {
    tracef("s-external interrupt from usertrap!");
    plic_handle();
    return 2;
} else {
    return 0;
}
}
}

```

调度算法--Round Robin

在 `sched.c` 中，我们会有一个队列 `task_queue` 管理状态是 `RUNNABLE` 的进程，这个链表我们称之为就绪队列。

xv6中，一个进程在以下时刻，会被设置成Runnable并且被 `add_task()` enqueue 进队尾。 - fork
- exec -> load_init_app - wakeup - yield -> sched -> scheduler

当scheduler通过 `fetch_task` 得到队头的就绪进程 `p` 后，进程会被移出就绪队列，并且状态改为 `RUNNING`。

由于我们每次将进程加入就绪队列是加在队尾，而选择下一个就绪进程是从队头，因此我们已经实现了Round_Robin调度算法。

```

//os/sched.c
void scheduler() {
    //.....

    for (;;) {
        // intr may be on here.

        p = fetch_task();
        //.....

        acquire(&p->lock);
        assert(p->state == RUNNABLE);
        debugf("switch to proc %d(%d)", p->index, p->pid);
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->sched_context, &p->context);

        // When we get back here, someone must have called swtch(..., &c->sched_context);
        assert(c->proc == p);
        assert(!intr_get()); // scheduler should never have intr_on()
        assert(holding(&p->lock)); // whoever switch to us must acquire p->lock
        c->proc = NULL;

        if (p->state == RUNNABLE) {
            add_task(p);
        }
        release(&p->lock);
    }
}

```

```
void sched() {
    //.....
    swtch(&p->context, &mycpu()->sched_context);
    //.....
}

// Give up the CPU for one scheduling round.
void yield() {
    struct proc *p = curr_proc();
    debugf("yield: (%d)%p", p->pid, p);

    // lab9 cpu scheduling:
    infof("yield: %d", p->pid);

    acquire(&p->lock);
    p->state = RUNNABLE;
    sched();
    release(&p->lock);
}
```

? Question

结合本周与 Week 5 的实验内容，总结时钟中断触发时从一个进程切换到另一个进程都会经过哪些进程的哪些函数。